



Chapter 3

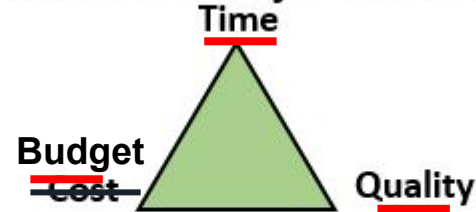
Quality Management & Assurance

Chapter 3

Quality Mgmt & Assurance

From chapter 1

Project managers need to face 3 major constraints:



Project managers' mantra:

To develop the system on time, within budget and to the required quality



Consequences of low quality ...

E.g. 1: Knight Capital's Faulty Trading Software (August 2012)

Knight Capital had to give a big payout for its buggy trading software: \$440 million. The software's algorithm was designed to enact automatic orders over several days but a **bug** caused the software to make all the orders in 1 hour, buying and selling stock with a loss of up to 15 cents per share. Ultimately, Knight Capital lost 4x its 2011 profit and was acquired by a **rival company** (Getco LLC) in Dec 2012.

Consequences of low quality ...

E.g. 2: Google's Nest's Smart Thermostat Software Bug

(January 2016)

A device used to **control temp** within a device.

- Activate **cooling** when temp rises too high
- Activate **heating** when temp falls below certain point

When a thermostat is deactivated, it will not be able to control the device temp.

E.g. unable to activate **cooling** when temp rises too high

A December software update contained a bug which deactivated the thermostat and drained the device's battery. Even when developers fixed the problem, they were left with annoyed customers who were less than happy to undertake the 9-step process to reboot the device ...

Consequences of low quality ...

E.g. 3: PayPal accidentally credits man \$92 quadrillion

By Sho Wills, CNN

Updated 1355 GMT (2155 HKT) July 17, 2013

CNN BUSINESS

Account Statement | June 2013

Reynolds, Christopher

Email (PayPal Account ID): [REDACTED]

PayPal

Statement period:

June 1, 2013 - June 30, 2013

Balance Summary*

	USD
Beginning Balance	140.25
Ending Balance	-92,233,720,368,547,800.00

Account Activity



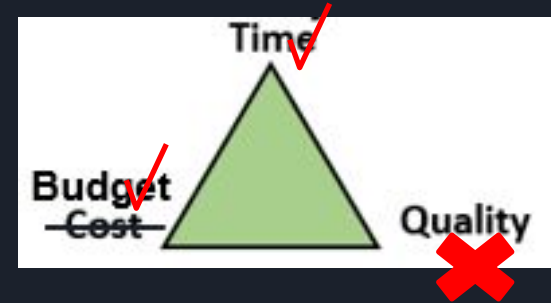
Software with Low Quality

Give 1 example _____

Software with Low Quality

E.g. 4:

Existing manual system is simple and straight forward [only needs 5-10 mins to fill-in the expense reimbursement form] but using the new computerised system it took much longer to do the same).





T.O.C

3.1 Quality Concepts & Principal **Activities**

3.2 Quality Planning & Quality Control

3.3 Process and Product Quality Relationship

3.4 Techniques to Help Enhance Software Quality

3.5 Quality Management and Costs

3.6 Quality Assurance and **Standards**



3.1 Quality Concepts & Principal Activities

- The Importance of Software Quality
- Defining Software Quality
- Principal Quality Activities
 - QA
 - QP
 - QC

The Importance of Software Quality

- Organizations are relying more on computer systems
 - high quality software will ...
- Most safety-critical systems have built-in software
 - high quality software will ensure that system functions as expected in ALL situations.
- Quality retains customers and increases profits
- Quality is essential for international marketing



Defining Software Quality

- Quality simply means **a product meet its specification.**
- The **subjective** quality of a software system is **largely** based on its non-functional characteristics. This reflects practical **user experience**

Appendix 3.1 ISO/IEC 25010 Quality Model & Attributes

Table 2 — Product Quality characteristics and subcharacteristics

	Reliability
	Maturity
	Availability
	Fault tolerance
	Recoverability
	Security
	Confidentiality
	Integrity
	Non-repudiation
	Accountability
	Authenticity
	Maintainability
	Modularity
	Reusability
	Analysability
	Modifiability
	Testability
	Portability
	Adaptability
	Installability
	Replaceability
Functional suitability	
Functional completeness	
Functional correctness	
Functional appropriateness	
Performance efficiency	
Time behaviour	
Resource utilization	
Capacity	
Compatibility	
Co-existence	
Interoperability	
Usability	
Appropriateness recognizability	
Learnability	
Operability	
User error protection	
User interface aesthetics	
Accessibility	

Software Quality Attributes

- Quality simply means **a product meet its specification.**
- The subjective quality of a software system is largely based on its non-functional characteristics. This reflects practical user experience

When planning for quality, select the most important quality attributes for the software being developed.

Appendix 3.1 ISO/IEC 25010 Quality Model & Attributes

Table 2 — Product Quality characteristics and subcharacteristics

	Reliability
	Maturity
	✓ Availability
	Fault tolerance
	Recoverability
	Security
	Confidentiality
	Integrity
	Non-repudiation
	Accountability
	Authenticity
	Maintainability
	Modularity
	Reusability
	Analysability
	Modifiability
	Testability
	Portability
	Adaptability
	Installability
	Replaceability
Functional suitability	
Functional completeness	
Functional correctness	
Functional appropriateness	
Performance efficiency	
✓ Time behaviour	
Resource utilization	
Capacity	
Compatibility	
Co-existence	
Interoperability	
Usability	
Appropriateness recognizability	
✓ Learnability	
Operability	
✓ User error protection	
✓ User interface aesthetics	
Accessibility	

Q) Which quality attributes are important for an e-commerce website?

Q) Can you suggest one more _____?

Software Quality Attributes

- Quality simply means a product meet its specification.
- The subjective quality of a software system is largely based on its non-functional characteristics. This reflects practical user experience

When planning for quality, select the most important quality attributes for the software being developed.

Conflicting Software Quality Attributes

- It is not possible for any system to be optimized for all of these attributes

E.g., **increasing** SECURITY may lead to **lower** performance

- There is a tension between **customer** quality requirements (efficiency, reliability, etc.) and **developer** quality requirements (maintainability, Portability)

Appendix 3.1 ISO/IEC 25010 Quality Model & Attributes

Table 2 — Product Quality characteristics and subcharacteristics

		Reliability
		Maturity
		Availability
		Fault tolerance
		Recoverability
		Security
		Confidentiality
		Integrity
		Non-repudiation
		Accountability
		Authenticity
		Maintainability
		Modularity
		Reusability
		Analysability
		Modifiability
		Testability
		Portability
		Adaptability
		Installability
		Replaceability
Functional suitability		
Functional completeness		
Functional correctness		
Functional appropriateness		
Performance efficiency		
Time behaviour		
Resource utilization		
Capacity		
Compatibility		
Co-existence		
Interoperability		
Usability		
Appropriateness recognizability		
Learnability		
Operability		
User error protection		
User interface aesthetics		
Accessibility		

TO ACHIEVE A
HIGH LVL OF
**PERFORMANCE
EFFICIENCY AND
HIGH LVL OF
SECURITY AT
THE SAME = NO**

When planning for quality, select the most important quality attributes for the software being developed then develop a quality specification for the product!

Desired quality attribute

Software Quality Specification

Qual.Attri.	:	<u>Define the quality attribute.</u>
Scale	:	Specify the <u>unit of measurement for the quality attribute</u>
Test	:	Describe the <u>method or process used to measure quality attribute</u>
Min/Max acceptable	:	Specify the <u>lowest/highest acceptable value for the quality attribute</u> . Values below/above this threshold means the product should be rejected.
<u>Target range</u>	:	Specify the <u>desired range of values for the quality attribute</u>
<u>Now</u>	:	The <u>current value of the quality attribute.</u>

Desired quality attribute

quality attribute definition

Software Quality Specification (E.g. 1)

Maintainability (Modifiability)	<i>Degree to which a system can be effectively and efficiently modified without introducing defects or degrading existing product quality</i>
Scale	Specify the <u>unit of measurement</u> for the quality attribute (e.g length of variable name)
Test	Describe the <u>method or process</u> used to measure the quality attribute 1. Count the no of characters used in variable name (take a sample of 6 programs) 2. Calculate the average length (e.g.) $\frac{(2+3+3+4+4+8)}{6} = 24/6 = 4$
Min acceptable	Specify the lowest acceptable value for the quality attribute. Values below this threshold means the product should be rejected. Min acceptable length = 5 characters
Target range	Specify the <u>desired range of values</u> for the quality attribute Desired range = 6 – 10 characters
Now	The <u>current value</u> of the quality attribute. 4 characters

Q) Is the length of var name up to the desired level to achieve a high degree a modifiability?

(E.g. 2) Software Quality Specification

Desired quality attribute

quality attribute definition

Usability (User Error Protection)	Degree to which a system protects users against making errors.	
Scale	The <u>unit of measurement</u> for the quality attribute. % err.protection (0 to 100), 0% indicates no error protection and 100% means the system has complete protection against user errors	
Test	Describe the <u>method or process</u> used to measure the quality attribute. (i) Examine all input screens. (ii) Count no of input fields with constraints[A], (iii) no of input fields w/o constraints but with helpful labels or instructions[B] and no of input fields w/o constraints and w/o helpful labels or instructions[C]. (iv) Percentage of protection= $([A+B]/[A+B+C] * 100\%)$. (Note: A+B+C = total no of input fields in the sys)	
Min acceptable	Specify the lowest acceptable value for the quality attribute. Values below this threshold means the product should be rejected. Min acceptable = 80% of protection	
Target range	Specify the <u>desired range</u> of values for the quality attribute 90% - 100%. This is to enhance user satisfaction.	Q) Is the system ability to protect user's from making errors up to the desired level?
Now	The <u>current value</u> of the quality attribute.. 70%	

(E.g. 3a) Software Quality Specification

Q) Has the system(XYZ M.App) under construction met the quality requirement? _____

Usability (efficiency)	Definition of “efficiency” How <u>quickly</u> a task can be completed
Scale	Specify the <u>unit of measurement</u> for the quality attribute. <u>TIME</u> (in minutes)
Test	Describe the <u>method or process</u> used to measure the quality attribute 1. Run usability test with 10 users. 2. Record time each user took to pay for 2 hours of street parking using the XYZ Mobile App 3. Calculate the average time taken $\left(\frac{\text{Total time taken by 10 users}}{10} \right)$
max acceptable	Specify the highest acceptable value for the quality attribute. Values above this threshold means the product should be rejected. Max acceptable time = 1 min
Target range	Specify the <u>desired range</u> of values for the quality attribute 0.5 – 1 min This is to ensure that user will adopt the XYZ Mobile App
Now	The <u>current value</u> of the quality attribute. 1.5 mins (after 2 times of system redesign)

(E.g. 3b) Software Quality Specification

Q) Has the system(XYZ M.App) under construction met the quality requirement? _____

Usability (efficiency)	Definition of “efficiency” How <u>quickly</u> a task can be completed	<u>Purpose of SQS</u> to ensures the final product meets the <u>desired</u> quality standard
Scale	Specify the <u>unit of measurement</u> for the quality attribute. <u>TIME</u> taken (in minutes)	
Test	Describe the <u>method or process</u> used to measure the quality attribute 1. Run usability test with 10 users. 2. Record time each user took to pay for 2 hours of street parking using the XYZ Mobile App 3. Calculate the average time taken (<u>Total time taken by 10 users</u>) 10	
max acceptable	Specify the highest acceptable value for the quality attribute. Values above this threshold means the product should be rejected. Max acceptable time = 1 min	
Target range	Specify the <u>desired range</u> of values for the quality attribute 0.5 – 1 min This is to ensure that user will adopt the XYZ Mobile App	
Now	The <u>current value</u> of the quality attribute. 0.6 mins (after 3 times of system redesign)	

Revisit (E.g. 1)

Software Quality Specification

Maintainability (Modifiability)	<i>Degree to which a system can be effectively and efficiently modified without introducing defects or degrading existing product quality</i>
Scale	Specify the <u>unit of measurement</u> for the quality attribute (e.g.1 length of variable name) e.g.2 Depth of nested if statement
Test	Measurement will give you an INDICATOR on the “Degree to which a system can be effectively and efficiently modified without introducing defects or degrading existing product quality” Measure the quality attribute ble name (in all programs) +3+4+4+8) = 24/6 = 4) 6
Min/max acceptable	ie for the quality attribute. Values below/above this ected. 5 characters
Target range	Specify the <u>desired range of values</u> for the quality attribute 6 – 10 characters
Now	The <u>current value</u> of the quality attribute. 4 characters



Principal Quality Activities

- Software Quality Management (SQM)'s **3 main activities**:
 1. **Quality Assurance (QA)**: to develop an organizational framework (procedures & standards) that will lead to high quality of software.
 2. **Quality Planning (QP)**: to select appropriate procedures and standards from the framework and adapt to a specific software project.
 3. **Quality Control (QC)**: to execute processes to ensure that software development follows the quality procedures and standards.

- In project mgmt, the QA team should be independent from the development team so that they can take an objective view of the work products.



3.1 Quality Concepts & Principal Activities

- The Importance of Software Quality
- Defining Software Quality
 - developing a software quality specification for the product
- Principal Quality Activities
 - QA
 - QP
 - QC

3.2 Quality Planning & Quality Control

- Introduction
- Quality Plans
- Quality Control



Introduction

- QA establishes the infrastructure that supports solid software engineering methods, rational project management and quality control actions.
- QC encompasses a set of software engineering actions that help to ensure that each work product meets its quality goals.
- Some organizations produce quality plans for each project.



Quality Plans

A quality plan

- Specifies the desired product qualities and how these are assessed (e.g. IF usability is the desired quality, it may be assessed through user testing. Target = minimum 90% task success rate)
 - Should indicate which organizational standards should be applied (*and where necessary, define new standards to be used*)
-



Quality Plans Structure

- Product introduction
- Product plans
- Process descriptions
- Quality goals
- Risks and risk management

Note:

- quality plans should be short, succinct documents
- if they are too long, no one will read them.



Quality Control (QC)

- QC **involves monitoring the software development process** to ensure that QA procedures and standards are being followed.
- The deliverables from the software development process are checked against the defined project standards during the QC process, either using quality reviews and/or automated software assessment.



Quality Reviews

- Quality reviews involve a group of people checking part or all of a software process, system or its documentation **to find potential problems.**
- Outcome of the review
 - List of problems found
 - author to do corrections

Review Type	Purpose
Design or program reviews	To find errors in the requirements, design or code.
Progress reviews	To find out project progress and report to management
Quality reviews	To carry out a technical analysis of product components or documentation to find mismatches between the specification and the component design, code or documentation and to ensure that defined quality standards of the organization have been followed.



3.1 Quality Concepts & Principal Activities

3.2 Quality Planning & Quality Control

3.3 Process and Product Quality Relationship

3.4 Techniques to Help Enhance Software Quality

3.5 Quality Management and Costs

3.6 Quality Assurance and Standards

3.3 Process and Product Quality Relationship

- **In general, the quality of the development process directly affects the quality of delivered products.** The quality of the product can be measured and the process is improved until the proper quality level is achieved.
- In manufacturing systems, there is a clear relationship between the production process and product quality. However, quality of software is highly influenced by the **experience of software engineers**. In addition, it is difficult to measure certain software quality attributes and to tell how process characteristics influence these attributes



3.4 Techniques to Enhance Software Quality

- Inspections
- Cleanroom software development
- Software quality circles
- Lessons learnt report



Inspections

- Inspections can be applied to documents produced at any development stage.
- When a piece of work is completed, copies are distributed to co-workers who examine the work, noting defects/problems. A meeting then discusses the work and a list of defects requiring rework is produced.
- E.g. of works that may be examine:
 - Table design (e.g. No of fields in a table – too little?, too many?)
 - Program codes (e.g. length of variable names, _____, etc)
 - _____
- Inspections (aka Fagan method) was pioneered by an IBM employee

Benefits of Inspections



https://www.google.com/search?q=inspection+clipart&client=firefox-b-d&source=lnms&tbn=isc&sa=X&ved=2ahUKEwidoNfyxvj7AhXc-TgGHUo-DBwQ_AUoAXoECAIQAw&biw=1525&bih=687&dpr=0.9

- Very effective in removing superficial errors. (e.g. Register, Sign-up For An Account)
- It can enhance team spirit
- Helps spread good programming practices as the participants discuss specific pieces of code.
- Helps developers to produce better structured and self-explanatory software.

Principles of the Fagan method

- Inspections are **carried out on all major deliverables.**
- All types of defect are noted - not just logic or function errors.
- Inspections **can be carried out by colleagues at all levels except the very top.**
- Inspection meetings do not last for more than 2 hours.
- Inspections are carried out using a predefined set of steps.
- The inspection is **led by a moderator who has been trained in the technique.**
- The other participants have defined roles (e.g., one person will act as a recorder and note all defects found, and another will act as reader and take the other participants through the document under inspection).
- **Have a checklists in the fault-finding process.**
- Material is inspected at an optimal rate of about 100 lines an hour.
- Statistics are maintained so that the effectiveness of the process can be monitored.



Cleanroom Software Development

- Improved from structured programming: de-component complex system so that each component has only one entry and exit point which make testing easier and more precise.
- Aims to *avoid defect* rather than detect and repair them:
 - Use incremental development approach
 - Use statistical testing/verification
- Rationale for incremental approach:
 - User requirement changes are inevitable
 - New requirements are added later; Main modules given first (gradually increase the implemented functionality)
 - Each new function will be tested and measured against pre-defined standards. If fail to meet the standards, go back to design phase.



Cleanroom Software Development

3 separate teams:

- **Specification team**
 - obtains the user requirements and a *usage profile* estimating the volume of use for each feature in the system
- **Development team**
 - develops the code in increments but does not do testing on the program code produced
- **Certification team**
 - carries out testing which is continued until a statistical model shows that the failure intensity has been reduced to an acceptable rate

Cleanroom Benefits & Weaknesses

Benefits

- **Lower number of errors**
(codes are develop by dev-team but testing are carried out by certification-team)
- **Software of higher quality**
(lower no of errors means higher software quality)
- **Lower cost**
(lower no of errors also means there will be less rework, less rework is a cost saving)
- **Project on schedule**
due to less rework

Weaknesses

- Works well with skilled and committed engineers only.
- This approach is confined to a few technologically advanced organizations only.



Software Quality Circle

- The aim of Japanese quality circle approach is to examine and modify the activities in the development process in order to reduce the number of errors that they have in their end-products.
- Testing and Fagan inspections can assist the removal of errors, but the same types of error could occur repeatedly in successive products created by a faulty process. By uncovering the source of errors, this repetition can be eliminated.

Quality Circle



https://www.google.com/search?q=quality+circle+clipart&tbm=isch&ved=2ahUKEwjroakzP7AixV_TgH5SkFDyIO2-cCgQIABAA&oeq=quality+circle+clipart&gs_lcp=CgNpbWcoQAT1DeCvjaCWDIC2gAcAB4IAIBNYgBaZIBATKYAOQgAQGgAQInd3Mtd2l6LWlZ8ABAQ&scit=im&ei=-liaY-zSHdX74-EPqYq8kAI&bih=687&biw=1525&client=firefox-b-d

- A faulty process will repeatedly produce faulty products.
- Quality Circle is to **FIX THE PROCESS** so that the same types of error will not occur again

- A quality circle is a group of 4 to 10 volunteers working in the same area (e.g. software development dept). They meet for about an hour a week to identify, analyze and solve their work-related problems.
 - One volunteer is the group leader.
 - There could be an outsider, a facilitator, who can advise on procedural matters.
- In order to make the quality circle work effectively, training needs to be given.
- **Together, the group**
 - selects a pressing problem that affects their work;
 - Identify the cause of the problem, and
 - Decide on a course of action to remove the problems.



Lessons Learnt Reports

- **Written by the project manager** as soon as possible after the completion of the project.
- Includes
 - **reflect on the performance of the recently completed project** when the experience is still fresh, and
 - **identify lessons to be applied to future projects.**
- However, one frequent problem is there is often very little follow-up on the recommendations of such reports, as there is often nobody within the organization with the responsibility and authority to do so.



Quick Review...

3.4 4 Techniques to Enhance/Improve Software Quality

- Inspections
- Cleanroom software development
- Software quality circles
- _____



3.5 Quality Management & Costs

- Quality Management
- Costs of Quality
- Categories of Quality Costs
-



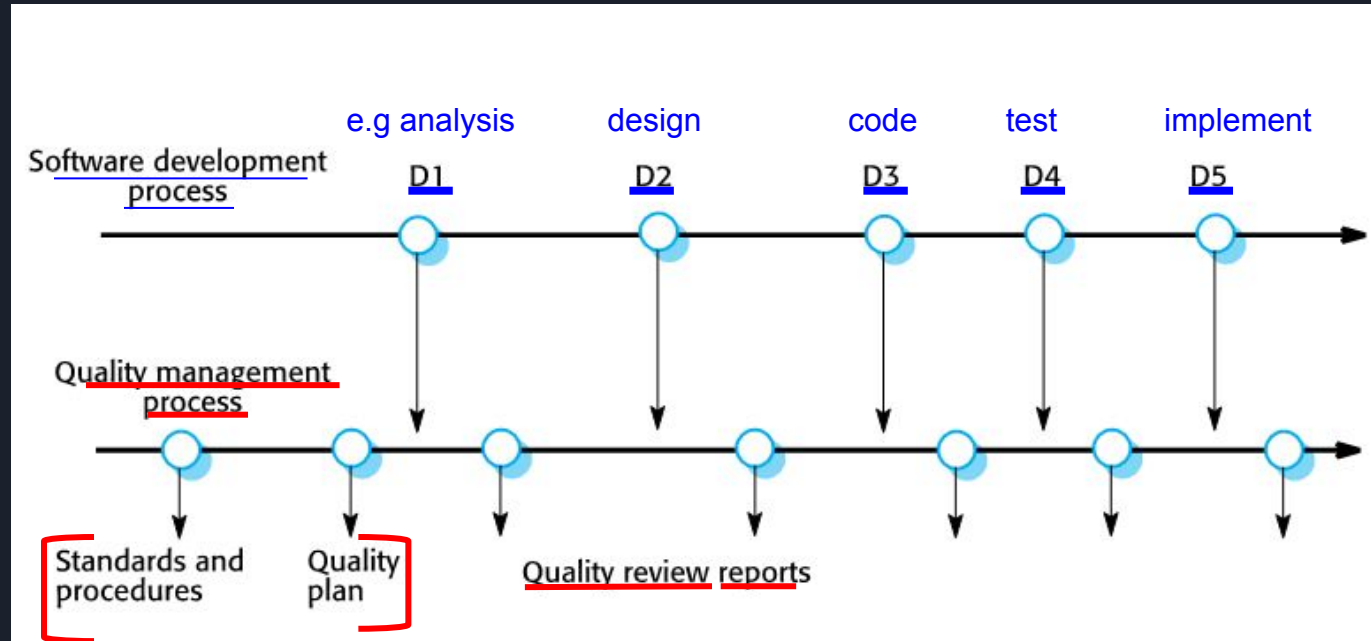
Quality Management

is to ensure that the required level of quality is achieved in a software product.

- At the organizational level, it is concerned with establishing a framework of organizational processes and standards that will lead to high-quality software.
- At the project level, quality management
 - involves the application of specific quality processes and checking that these processes have been followed.
 - concerned with establishing a quality plan for a project. The quality plan should set out the quality goals for the project and define what processes and standards are to be used

Quality Management (cont'd)

- **Quality management processes** are to check on the software development process.



... to ensure that standards and procedures are followed and the product meets quality requirements



Costs of Quality

Costs of
quality

= **Cost of achieving
quality** + **Cost of low quality
software**

*(costs incurred in
performing
quality-related
activities
e.g. _____)*

examples

- *Rework to correct an error*
- *Resolve complaint*
- *Provide customer support*
- *etc*



Categories of Quality Cost

The costs of quality can be divided into

- a. Prevention Costs
- b. Appraisal Costs
- c. Failure Costs



Categories of Quality Cost

Prevention Costs

Costs related to performing:

- **mgmt activities** - To plan and coordinate all QA and QC activities
- **technical activities** - To develop complete requirements & design models
- To **conduct training** associated with the above activities

Appraisal Costs

Costs to:

- Conduct **technical reviews**
- **Test** products
- **Collect** data and report inspection

The goal of all these activities are to ensure that a project is error-free or within an acceptable range

c. Failure Costs

Internal Failure costs

Examples

- Costs to correct design errors
- Costs to correct program errors

Costs incurred prior to product delivery

External Failure costs

Examples

- Resolving customer complaint
- Handling product return and replacement
- Providing help line support
- Managing poor reputation and loss of business

Costs incurred after the product has been delivered to customers

Categories of Quality Cost

Costs of quality:

a. Prevention Costs

- Management activities - to plan and coordinate all QA and QC activities
- Technical activities - to develop complete requirements and design models
- To conduct training associated with these activities

b. Appraisal Costs

- Conduct technical reviews
- Test products
- **Internal Failure costs**
- Collect data and report inspection
- Costs to correct design errors, Costs to correct program errors

c. Failure Costs

- Resolving customer complaint, Handling product return and replacement, Providing help line support, Managing poor reputation and loss of business

Q) Which cost is the most costly/expensive to a software company?

How to avoid “Failure Costs”?



to develop an organisational framework (**procedures & standards**) that will lead to high quality of software.

3.6 Quality Assurance & Standards

- Quality Standards
- ISO/IEC 25010:2011
- ISO 9001 Standards Framework
- Quality Culture



Quality Standards

Quality standards are sets of guidelines, systems, methods, requirements, and specifications followed by an organization to ensure consistent process, product and service quality.

- Quality standards define the required quality of a product or process.
- Standards play an important role in quality management.
- **Standards may be international, national, organizational or project standards.**

The Importance of Quality Standards

- To ensure that products, processes and services meet customer expectations and/or comply with regulatory requirements.
- Encapsulation of best practice - avoids repetition of past mistakes.
- They are a framework for defining what quality means in a particular setting, i.e. the organization's view of quality.
- They provide continuity - new staff can understand the organization by understanding the standards that are used.



Product & Process Standards

- **Process standards** - define the processes that should be followed during software development
 - Include definitions of specification, design and validation processes, process support tools and a description of the documents that should be written during these processes.
- **Product standards** - apply to the software product being developed *including its documentation*
 - Include document standards (e.g. the structure of requirements documents, documentation standards, coding standards)

Product & Process Standards

Process Standards <i>(processes that should be followed during software development)</i>	Product Standards <i>(standards that apply to the software product being developed incl. its documentation)</i>
Conduct design <u>review</u>	Design <u>review form</u>
Submission of new code	Requirements document structure
Version release process	Method header format
Project plan approval process	Java programming style
<u>Test</u> recording process	Project plan format
Change control process	Change request form



Problems with Standards

- They often involve a lot of bureaucratic form filling
- They may NOT seen to be relevant and up-to-date by software engineers
- If they are unsupported by software tools, tedious form filling is often involved to maintain the documentation associated with the standards



Standards Development

- Involve practitioners in development - engineers should understand the rationale underlying a standards.
- Review standards and their usage regularly - standards that are outdated will have reduced credibility among practitioners
- Detailed standards should have specialized tool support - excessive clerical work is the most significant complaint against standards.



ISO/IEC 25010:2011

The ISO/IEC 25010:2011 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System & software quality models defines

- A quality in use model composed of 5 characteristics* that relate to the outcome of interaction when a product is used in a particular context of use.
- A product quality model composed of 8 characteristics* that relate to static properties of software and dynamic properties of the computer system

**some of which are further subdivided into sub-characteristics*

Note: this standard replaced the ISO/IEC 9126 Software engineering-Product quality standard. 55



ISO/IEC 25010:2011 can benefit these activities:

- Identifying software & system requirements
- Validating the comprehensiveness of a requirements definition
- Identifying software and system design objectives
- Identifying software and system testing objectives
- Identifying quality control criteria as part of QA
- Identifying acceptance criteria for a software product
- Establishing measures of quality characteristics in support of these activities



ISO/IEC 25010:2011 Quality in Use Model

- Effectiveness
- Efficiency
- Satisfaction
 - Usefulness
 - Trust
 - Pleasure
 - Comfort
- Freedom from risk
 - Economic risk mitigation
 - Health & safety risk mitigation
 - Environmental risk mitigation
- Context coverage
 - Context completeness
 - Flexibility

Refer to <https://www.iso.org/obp/ui/#iso:std:iso-iec:25010:ed-1:v1:en> for details.



The ISO 9001 Standards Framework

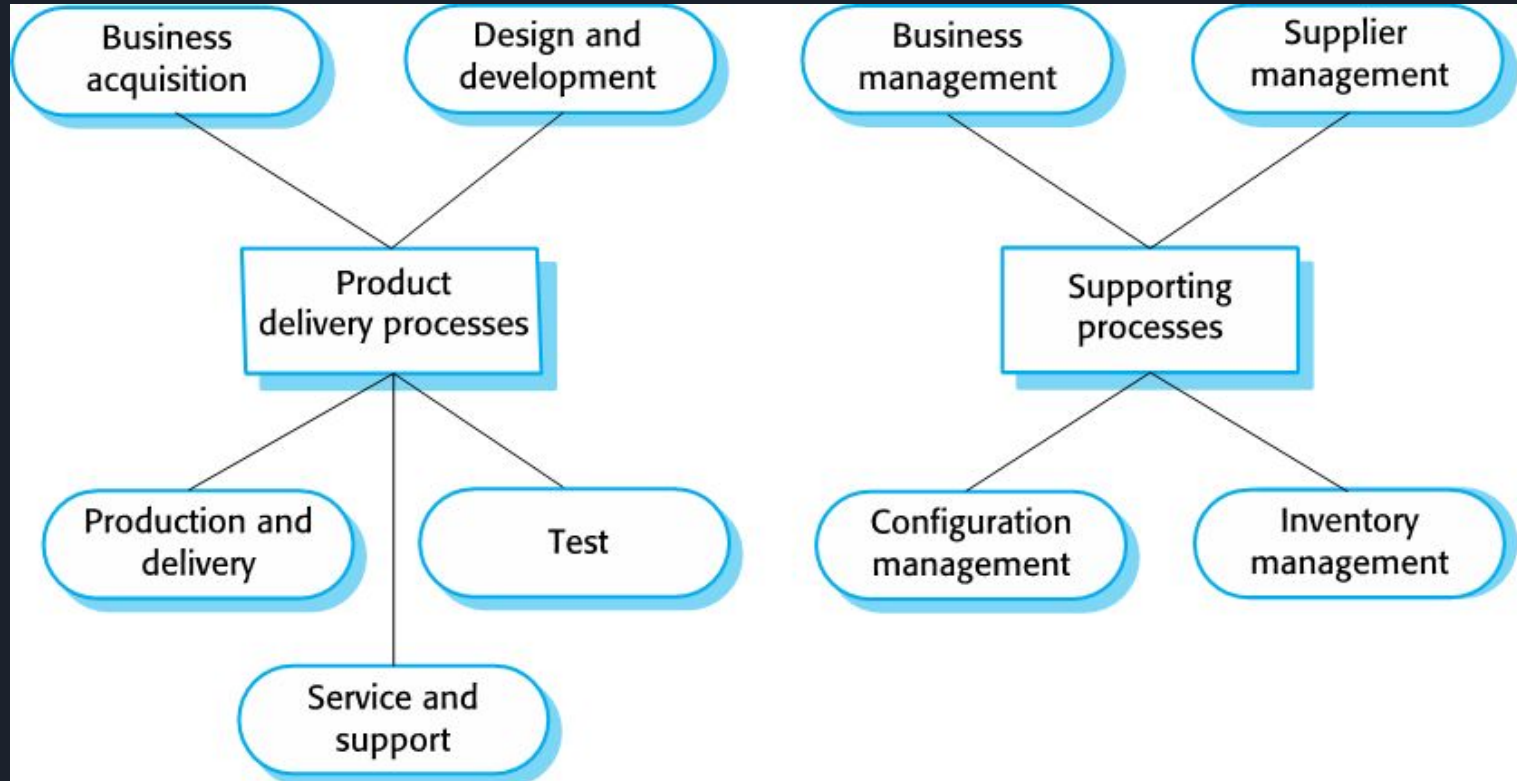
- An international set of standards that can be used as a basis for developing quality management systems.
- ISO 9001, the most general of these standards, applies to organizations that design, develop and maintain products, including software.
 - It sets out general quality principles, describes quality processes in general and lays out the organizational standards and procedures that should be defined. These should be documented in an organizational quality manual.



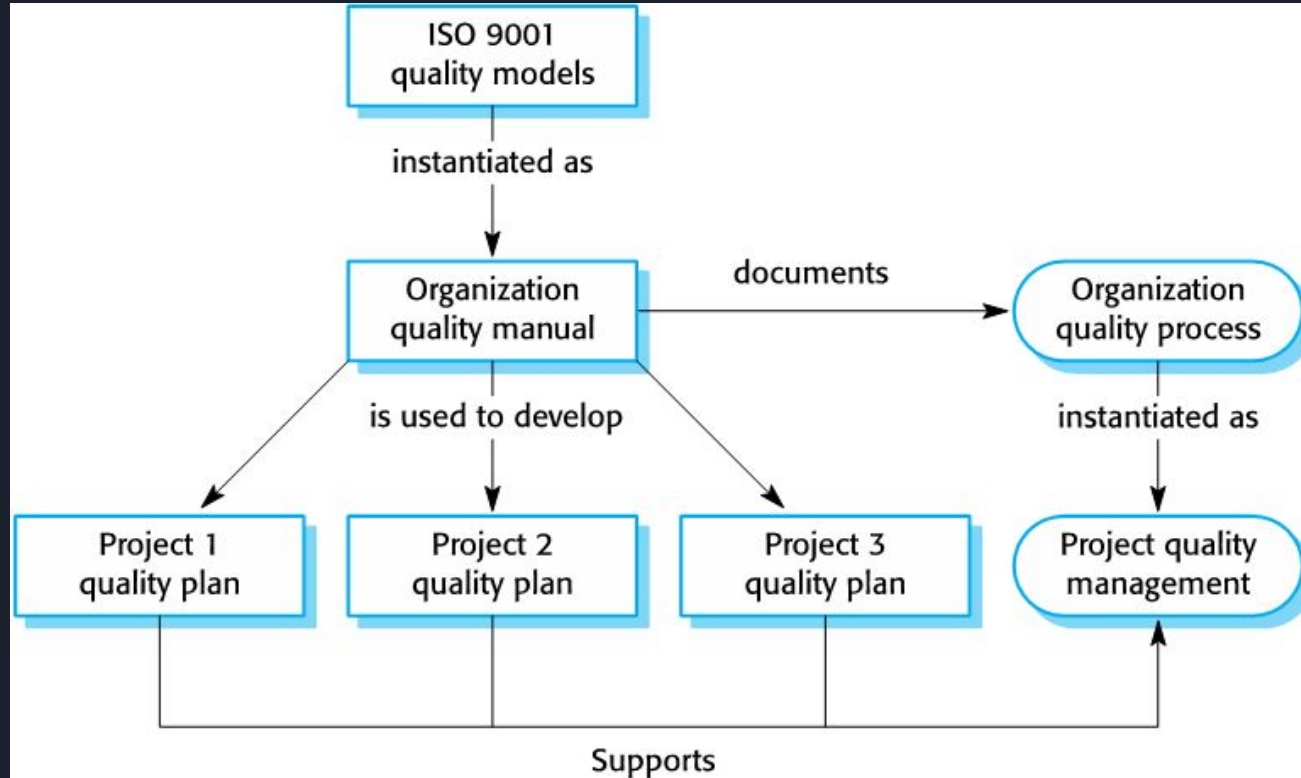
ISO 9001 Characteristics

- ISO 9001 is *not itself a standard for software development* but is a framework for developing software standards
- It sets *general quality principles*, describes quality processes in general, lays out the organizational standards and procedures that should be defined. These must be documented in an organizational quality manual.
- To be conformant with ISO 9001, a company must have defined the types of processes shown in the following figure and procedures that demonstrate that its quality processes are being followed.

ISO 9001 Core Processes



ISO 9001 & Quality Management





ISO 9001 Certification

- **Quality standards and procedures should be documented in an organizational quality manual.**
- An external body may certify that an organization's quality manual conforms to ISO 9000 standards.
- Some customers require suppliers to be ISO 9000 certified.



Quality Culture

- Organizations should aim to develop a quality culture where everyone responsible for software development is committed to achieving a high level of product quality.
- Teams should be encouraged to take responsibility for the quality of their work and to develop new approaches to quality improvement.
- They should support people who are interested in the intangible aspects of quality and encourage professional behavior in all team members.



Summary

3.1 Quality Concepts & Principal Activities

3.2 Quality Planning & Quality Control

3.3 Process and Product Quality Relationship

3.4 Techniques to Help Enhance Software Quality

3.5 Quality Management and Costs

3.6 Quality Assurance and Standards